

Neuro-symbolic approaches to the SAT problem

INTELLIGENT SYSTEMS FOR PATTERN RECOGNITION
REINFORCEMENT LEARNING

University of Pisa

survey with coding project

Donato Meoli

The SAT problem

- ▶ **SAT**: given a Boolean formula in CNF $\varphi = \bigwedge_i (\bigvee_j \ell_{ij})$, is there an assignment of the variables that makes it true? First problem proven NP-complete; the engine of verification, planning, synthesis.
- ▶ Modern solvers are **CDCL**: *decide* a variable, *unit-propagate*, on a *conflict* learn a clause and *backtrack*.
- ▶ The **branching heuristic** (which variable to decide next, e.g. VSIDS) drives performance — yet it makes poor choices during its warm-up phase, when activity scores are still uninformative.
- ▶ *This project*: **learn the branching heuristic** with two neuro-symbolic methods, modernise them to PyTorch, study **where graph attention helps**, and reproduce the published results.

Roadmap

Two ways to learn a SAT heuristic:

- ▶ **Graph-Q-SAT** — value-based RL (DQN) over a *graph* neural network on the CNF graph. Size-agnostic.
- ▶ **AlphaZeroSAT** — self-play + MCTS over a *CNN* policy/value net on the clause $\times$ variable matrix. Fixed size.

Our contributions:

- ▶ port both from TensorFlow 1.x + GSL to one PyTorch / PyTorch-Geometric stack; reproduce the published numbers *exactly*;
- ▶ **GAT-Q-SAT**: an attention-augmented successor;
- ▶ **restricted heuristics** for online speed-ups.

Plan: background on the CNF graph $\rightarrow$ Graph-Q-SAT & GAT $\rightarrow$ AlphaZeroSAT $\rightarrow$ implementation $\rightarrow$ results.

- ▶ The CNF as a graph
- ▶ Message Passing Neural Networks (MPNNs)
- ▶ NeuroSAT — the seminal graph model for SAT

(ISPR – survey)

Gilmer et al., *Neural Message Passing for Quantum Chemistry*.

Selsam et al., *Learning a SAT Solver from Single-Bit Supervision*.

Background:
learning on
the CNF graph

The CNF as a graph

A CNF is naturally a **bipartite graph**:

- ▶ a node for every **variable** (or literal) and every **clause**;
- ▶ an edge between a literal and each clause it occurs in (edge feature = **polarity**); for the literal view, an edge between complementary literals $x, \bar{x}$.

This representation is **permutation-invariant** and **size-agnostic**: the same network handles formulas of any size — the property the fixed-size CNN lacks. It is the substrate shared by NeuroSAT and Graph-Q-SAT.

$$\varphi = (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_3)$$

variables	clauses
x_1	
x_2	c_1
x_3	c_2

edges carry the literal polarity $\{+, -\}$.

Message Passing Neural Networks (MPNNs)

A general framework for learning on graphs (Gilmer et al., 2017). Each node v holds a hidden state h_v^t ; for T rounds it exchanges messages with its neighbours $\mathcal{N}(v)$:

$$\text{message: } m_v^{t+1} = \sum_{w \in \mathcal{N}(v)} M_t(h_v^t, h_w^t, e_{vw})$$

$$\text{update: } h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

$$\text{readout: } \hat{y} = R(\{h_v^T : v \in G\})$$

- ▶ M_t (message), U_t (update, often a GRU/LSTM) and R (readout) are learned and *shared across nodes and rounds*.
- ▶ GNNs, gated GNNs, GCN, GAT and the encode-process-decode network are all instances of this template.

NeuroSAT — a SAT solver from single-bit supervision

Literal/clause embeddings $L^{(t)} \in \mathbb{R}^{2n \times d}$, $C^{(t)} \in \mathbb{R}^{m \times d}$;
bipartite adjacency M ; MLPs $L_{\text{msg}}, C_{\text{msg}}, L_{\text{vote}}$; LSTM
updates L_u, C_u . One round:

$$(C^{(t+1)}, C_h^{(t+1)}) \leftarrow C_u([C_h^{(t)}, M^\top L_{\text{msg}}(L^{(t)})])$$
$$(L^{(t+1)}, L_h^{(t+1)}) \leftarrow L_u([L_h^{(t)}, F(L^{(t)}), M C_{\text{msg}}(C^{(t+1)})])$$

after T rounds, vote and average:

$$L_*^{(T)} \leftarrow L_{\text{vote}}(L^{(T)}), \quad y^{(T)} \leftarrow \text{mean}(L_*^{(T)}).$$

F swaps each literal with its complement.

- ▶ Trained *only* as a SAT/UNSAT classifier (one bit per formula).
- ▶ Yet it **learns to find satisfying assignments**: it guesses UNSAT with low confidence until a solution crystallises in the literal votes.
- ▶ Establishes the **graph view** of a CNF — the foundation for Graph-Q-SAT.
- ▶ *Limitation*: a standalone solver, not competitive in wall-clock with CDCL.

- ▶ Reinforcement learning for branching
- ▶ Background
 - ▶ Deep Q-Learning (DQN)
 - ▶ Graph Networks
- ▶ Model — encode—process—decode
- ▶ Graph Attention (GAT)
- ▶ Results

(ISPR + RL – survey with extra coding)

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*

Graph-Q-SAT & GAT-Q-SAT

Branching as a Markov Decision Process

Graph-Q-SAT turns the CDCL search into an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- ▶ **state** s : the current CNF as a bipartite graph (variable/clause nodes, polarity edges, a global attribute);
- ▶ **action** a : assign an unassigned variable True/False — one decision;
- ▶ **reward** $r = -p$ at every non-terminal step, 0 at termination $\Rightarrow$ maximising the return *minimises the number of decisions*;
- ▶ **return** $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$.

Goal: learn the action-value (*quality*) function

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \right] \quad (\text{Bellman optimality}).$$

The branching heuristic is then $\pi(s) = \arg \max_a Q^*(s, a)$ over the variable nodes.

Deep Q-Learning (DQN)

Approximate Q^* with a network Q_θ , fitted by minimising the temporal-difference error over a replay buffer $\mathcal{D}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\underbrace{r + \gamma \max_{a'} Q_{\theta^-}(s', a')}_{\text{TD target}} - Q_\theta(s, a) \right)^2 \right]$$

- ▶ **target network** θ^- (a slow copy of θ) stabilises the target;
- ▶ **experience replay** $\mathcal{D}$ breaks temporal correlations;
- ▶ **ϵ -greedy** exploration: random action w.p. ϵ , else $\arg \max_a Q_\theta$.

Here the action set is *variable-sized* (one per unassigned variable), so Q_θ must be a **graph** network that scores each variable node.

Model — the encode–process–decode graph network

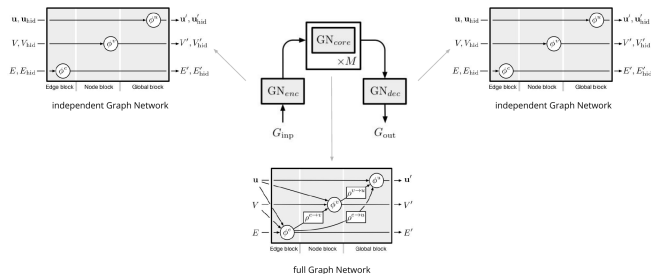
A graph-network block (Battaglia et al.) updates edges, then nodes, then the global:

$$e'_k = \phi^e(e_k, v_{r_k}, v_{s_k}, u)$$

$$v'_i = \phi^v(\rho^{e \rightarrow v}(\{e'_k\}), v_i, u)$$

$$u' = \phi^u(\rho^{e \rightarrow u}(\{e'_k\}), \rho^{v \rightarrow u}(\{v'_i\}), u)$$

ϕ are MLPs; ρ are permutation-invariant aggregations (sum). **Encode** (independent) $\rightarrow$ **core** ($\times M$ message-passing steps, shared) $\rightarrow$ **decode**; the per-variable Q -values come out of the decoder.



encode–process–decode: $GN_{enc} \rightarrow GN_{core} \times M \rightarrow GN_{dec}$.

Configuration & metric

How the model is configured:

- ▶ node features: 2-D one-hot (variable / clause); edge feature: polarity; global: empty.
- ▶ encoder/core/decoder hidden size 64; M core message-passing steps (a key knob at test time);
- ▶ DQN: Adam, target-net sync, prioritised replay, $\gamma \approx 0.99$.

Metric — MRIR:

$$\text{MRIR} = \text{median} \frac{\#iters_{\text{MiniSat}}}{\#iters_{\text{model}}}$$

Median Relative Iteration Reduction: > 1 means *fewer* CDCL iterations than MiniSat's VSIDS. Reported as a function of the test-time decision budget (the cap).

Improvement idea: graph attention

- ▶ In the core block, a node aggregates *all* neighbours with equal weight ($\rho = \text{sum}$). But not all neighbouring clauses/variables are equally relevant to a branching decision.
- ▶ **Idea:** replace the uniform aggregation with a **learned, edge-aware attention** over neighbours, so the network can encode more abstract dependencies in the SAT graph.
- ▶ This is the *non-local* graph-network variant — a **Graph Attention Network (GAT)** inside the core.

Veličković et al., *Graph Attention Networks* (ICLR 2018).

Graph Attention Networks (GAT)

For node i with neighbours $\mathcal{N}_i$ and a shared linear map W , attention weights a message by relevance:

$$\text{score: } e_{ij} = \text{LeakyReLU}(\mathbf{a}^\top [Wh_i \parallel Wh_j])$$

$$\text{normalise: } \alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}$$

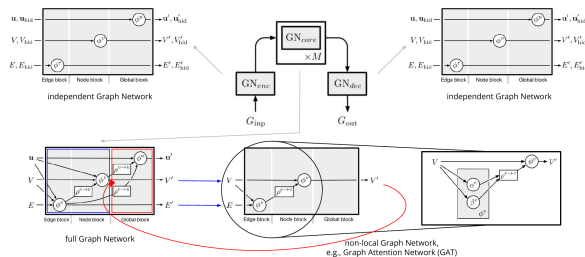
$$\text{aggregate: } h'_i = \sigma\left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} Wh_j\right)$$

- **multi-head:** K independent attentions are concatenated (or averaged) for stability and capacity;
- edges carry the literal polarity, so attention is **edge-featured**.

GAT-Q-SAT — our attention successor

GAT-Q-SAT (this work): replace the uniform neighbour aggregation in the core block with two edge-aware, multi-head GATConv layers (the *non-local* block, right), so the network can weight the most relevant neighbouring clauses/variables.

lineage: **Graph-Q-SAT** $\rightarrow$ **GAT-Q-SAT**.



the core becomes a *non-local* GN block — a GAT with edge attention α^e, β^e .

- ▶ Search-based RL for branching
- ▶ Background
 - ▶ Monte Carlo Tree Search (MCTS)
 - ▶ AlphaZero self-play
- ▶ Model — CNN policy/value net
- ▶ Results

(ISPR + RL – survey with coding)

Wang & Rompf, *From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha(Go) Zero*.

AlphaZeroSAT

Monte Carlo Tree Search (MCTS)

Treat solving as a game: each node is a partial assignment, each move a branching literal. From the root, repeat four steps:

1. **select** the child maximising the PUCT score

$$a_t = \arg \max_a \left(Q(s, a) + \underbrace{c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}}_{U(s, a)} \right);$$

2. **expand** a leaf and evaluate it with the network $(P, v) = f_{\theta}(s)$;
3. **simulate** / bootstrap from the value v ;
4. **back-up**: $N += 1$, $W += v$, $Q = W/N$.

The search yields an improved policy from the visit counts: $\pi(a \mid s) \propto N(s, a)^{1/\tau}$.

AlphaZero — self-play policy iteration

A single network $(p, v) = f_{\theta}(s)$ outputs a **policy** p over moves and a **value** $v \in [-1, 1]$ (probability of solving from s).

- ▶ **Self-play:** MCTS, guided by f_{θ} , produces a stronger policy π ; play it to the end to obtain an outcome z .
- ▶ **Learn:** regress $v \rightarrow z$ and $p \rightarrow \pi$ — MCTS is the policy-improvement operator, the network is the policy-evaluation operator:

$$\ell = \underbrace{(z - v)^2}_{\text{value}} \quad \underbrace{- \pi^{\top} \log p}_{\text{policy (cross-entropy)}} \quad + \quad \underbrace{c \|\theta\|^2}_{\text{regularisation}} .$$

- ▶ No human data, no heuristics: the heuristic *emerges* from self-play.

AlphaZeroSAT — the model

State: the CNF as a fixed-size **clauses** $\times$ **variables** matrix (entries $\{+1, 0, -1\}$ by literal polarity, as image channels).

Network: a CNN trunk (conv $32@8\times8$, $64@4\times4$, $64@3\times3$) then two heads:

$$\text{policy } p = \text{softmax}(\text{logits} + (\text{mask} - 1) \cdot \infty) \in \Delta^{2n}$$

$$\text{value } v = \tanh(\cdot) \in [-1, 1]$$

invalid literals (variables absent from remaining clauses) are masked to $-\infty$.

Model1/2/3 differ in the heads (fully-connected vs convolutional policy/value), following the original Alpha(Go)Zero-for-SAT design.

Env: MiniSat is extended into an MCTS *game* supporting hypothetical simulations.

Metric: mean branching *decisions* to solve (lower is better), reproducing the paper's Fig. 2.

Result (uf20-91, self-play): ≈ 5.4 mean decisions (best 3.4), down from ~ 15 for an untrained net.

But wall-clock: each decision is an MCTS search (≈ 160 ms/CPU) $\Rightarrow \approx 0.9$ s/problem vs MiniSat's **ms** — same time caveat as the graph models.

Limitation: the fixed-size matrix means the CNN **cannot generalise across problem sizes** — precisely the motivation for the graph methods.

- ▶ Modernisation & porting
- ▶ Restricted heuristics (online speed-ups)
- ▶ Reproduction (semantic non-regression)
- ▶ Where attention helps

(the coding project)

Implementation & Results

Modernisation — one self-contained PyTorch stack

- ▶ Ported **both** methods from **TensorFlow 1.x + GSL** to latest torch, torch-geometric, numpy $\geq$ 2, gymnasium — no torch-scatter/sparse, no GSL.
- ▶ Graph-Q-SAT: scatter drop-in, gym-robust env construction, a version-agnostic checkpoint loader (bridging the GATConv parameter renames across PyG versions).
- ▶ AlphaZeroSAT: re-implemented the CNN + AlphaZero loss in PyTorch; **removed GSL** (Dirichlet noise via C++11 `<random>`); trains end-to-end on GPU.
- ▶ Datasets unified into one shared data/ hub; both engines kept independent and runnable on Colab with Drive checkpointing.

Restricted heuristics — trading decisions for speed

Graph-Q-SAT cuts CDCL *iterations* but the GNN forward at every decision can lose on *wall-clock*. Three online tricks (Shirokikh et al. 2023), all added at eval time:

- ▶ **early release** (`-release_after N`): guide the first N decisions, then hand control back to MiniSat's VSIDS;
- ▶ **action pool** (`-action_pool_size K`): one GNN forward $\rightarrow$ top- K actions executed without re-running the net;
- ▶ **Q-value warm-start** (`-warmstart_release`): seed VSIDS activities from the root-state Q -values, $\text{activity}(x) = -1 / \max(Q(x, T), Q(x, F))$, then run pure VSIDS.

The model matters most at the cold start, where VSIDS is still blind.

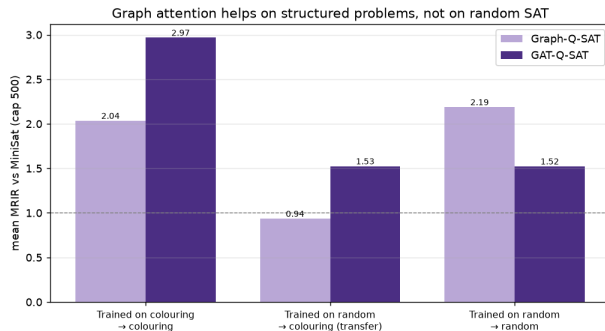
Reproduction (semantic non-regression)

- ▶ Re-ran the *original trained checkpoints* on the modern stack (latest torch / PyG / numpy ≥ 2 / gymnasium).
- ▶ MRIR matches the committed logs **exactly**:

Model	MRIR (median)	committed
Graph-Q-SAT	1.833	1.833
GAT-Q-SAT	1.667	1.667

- ▶ Drop-in changes only (scatter, env construction, GATConv key remap) — **semantics preserved**. Figures regenerated from the logs (no Sheets).

Results — the central finding

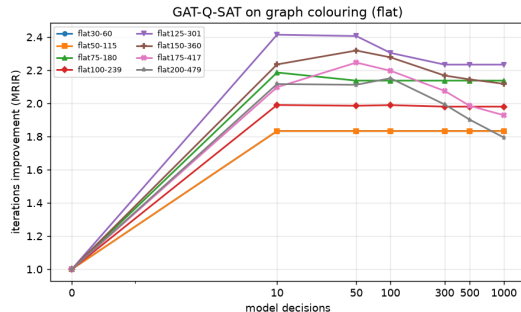
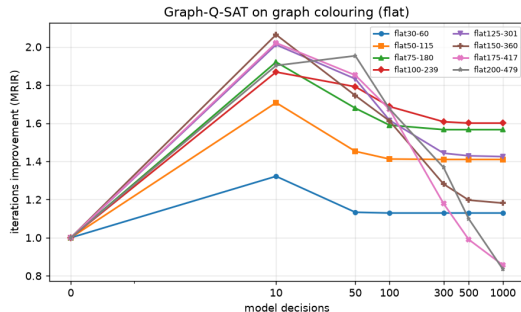


Attention helps on structured problems, not on uniform-random SAT.

Clean attention on/off ablation, matched training sets (mean MRIR, cap 500):

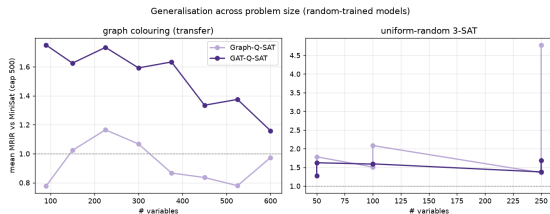
- ▶ **Colouring** (in-dist): GAT **2.97** vs 2.04.
- ▶ **Transfer** random→colouring: GAT **1.53** vs 0.94 (Graph-Q-SAT < 1 , i.e. worse than MiniSat!).
- ▶ **Random 3-SAT**: attention *does not* help (1.52 vs 2.19).

Results — MRIR curves on graph colouring



Iteration improvement vs the decision budget, one line per flat family — **GAT-Q-SAT lifts the whole curve on every size.**

Results — attention helps more on *larger* structured problems



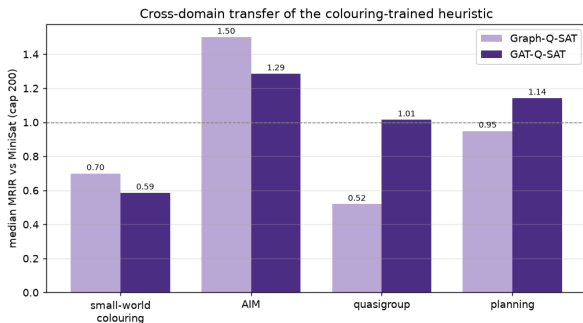
Random-trained: MRIR vs problem size (cap 500).

In-distribution graph colouring, MRIR per size:

set	Graph	GAT	Δ
flat30-60	1.83	2.00	+0.17
flat125-301	2.55	3.44	+0.88
flat150-360	1.92	3.76	+1.85
flat200-479	1.35	3.39	+ 2.04

Plain Graph-Q-SAT **degrades** on large instances; GAT-Q-SAT holds up — the advantage **grows with size**.

Results — cross-domain transfer (no retraining)

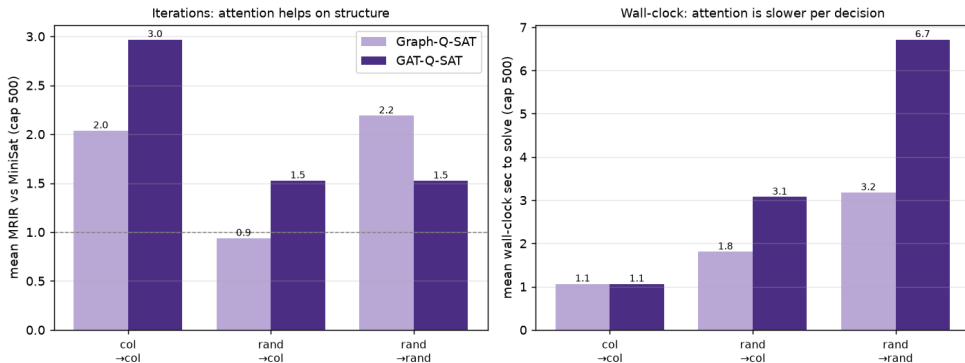


Setup: train on graph colouring, test *without retraining* on four *different* SAT families (small-world, AIM, quasigroups, planning); median MRIR, cap 200.

- ▶ **GAT-Q-SAT stays $\geq$ MiniSat on 3/4 domains; plain Graph-Q-SAT on 1/4.**
- ▶ **On the genuinely different domains GAT beats MiniSat, Graph drops below.**
- ▶ $\Rightarrow$ attention improves **cross-domain generalisation**.

Preliminary: quasigroup (11)/planning (8) small; single seed.

Results — fewer iterations $\neq$ less wall-clock time



Each decision is a **GNN forward**, so the iteration win (left) need not be a time win (right): on **col $\rightarrow$ col** GAT is fewer iters at the *same* 1.1s, but on **random** it is $\sim 2\times$ slower (6.7s vs 3.2s).
Absolute: 1–7s/problem vs MiniSat's **ms** — exactly what the **restricted heuristics** attack.

Where the field went & conclusions

- ▶ Lineage: NeuroSAT $\rightarrow$ NeuroCore $\rightarrow$ NeuroComb $\rightarrow$ **NeuroBack** (ICLR'24): a Graph *Transformer*, *offline* inference $\Rightarrow$ first wall-clock speed-ups on Kissat — **validating the attention direction** (GAT-Q-SAT).
- ▶ **Our results**: graph attention helps on structured problems (colouring), *growing with size*, and improves **cross-domain robustness** ($\text{GAT} \geq \text{MiniSat}$ on 3/4 unseen domains vs 1/4 for Graph).
- ▶ The fixed-size CNN (AlphaZeroSAT) cannot generalise across sizes — the motivation for the graph methods.

Code, report and reproducible figures: github.com/dmeoli/NeuroSAT.

Thank you for your
attention!

`github.com/dmeoli/NeuroSAT`